CONTROL-FLOW MODELS

Learning Goals

- produce intra-method control-flow models (i.e., flowcharts) from given Java source code
- produce inter-method control-flow models (i.e., call graphs) from given Java source code
- use a debugger to help produce control-flow models from given Java source code

Understanding Source Code

Control and data models help a software developer comprehend existing source code and can be used to help design new code. Software developers seldom write systems from scratch. Most often, software systems are long-lived and a software developer joining a team will be contributing to an existing system. Even when the construction of a new system begins, the system will typically be built using libraries of existing software components. As a result, software developers spend as much as half their time or more reading and understanding existing source code.

Software developers use many different techniques to help understand a part of a large source code base. In this reading, we will consider how developers use control-flow and data models to begin to understand source code. The models presented in this section of the course are intended to get you reading existing (simple) Java code. We will build and expand on these basic concepts in the remainder of the course.

Intermezzo: Models Revisited

In the last reading, we looked at how the source code itself is a model of the domain it addresses. In this chapter, we will consider a number of different kinds of models — for instance, flow charts, call graphs and data models — that can be extracted from an *existing* software system to help a software developer understand how the system works. As Figure 1 shows, these models can be extracted from the source code, by reading it, or from the executing system, by inspecting the system in a debugger.

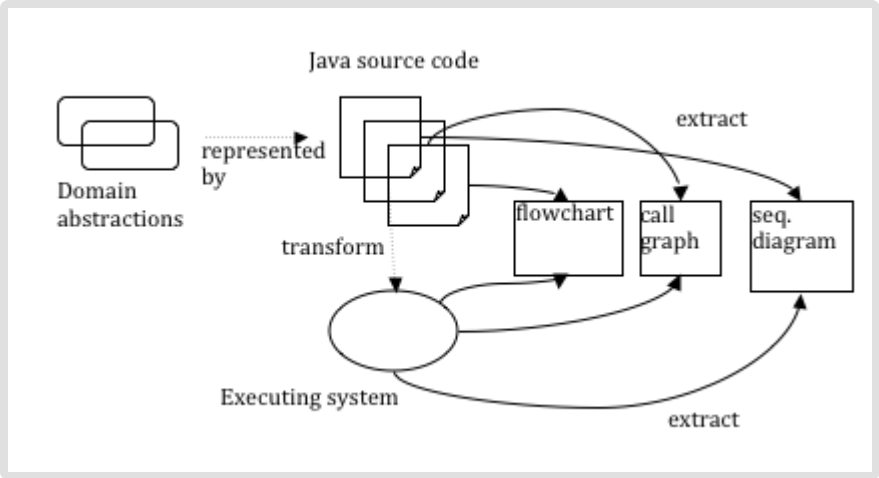


Figure 1: Models everywhere

In reality, the interaction between the models used for understanding the system and the models used to construct the system are not so clearly separated. With some programming systems, it is possible to generate a system or source code from the model. When models are used for generation, the approach is called model-oriented software engineering. A model-oriented approach is of particular interest to manufacturers of products who produce many variations of similar products, such as car manufacturers and cell phone makers, as the manufacturer can tweak the model and regenerate the typically large and complicated software system powering the device. We have not depicted these intricacies in Figure 1 for clarity.

We will start with a simple model that helps determine how to read code in one of the smallest units in a Java program, a method.

Control-Flow Models

In a Java program, recall that a method is the construct that describes a part of a computation to be performed. By calling methods in different ways and in different combinations, a program can support a range of computations and behave in different ways. The Java method shown below returns the value `true` if a specified year is a leap year, according to the Gregorian calendar, and the value `false` otherwise. The year of interest is passed to the method as a parameter named `year`.

```
/*
 * Determine whether a given year is a leap year according to the
 * Gregorian calendar.
 */
boolean isLeapYear(int year) {
    // declare a variable to store the value to be returned
    boolean isLeap = false;

    // if the year is divisible by 4 and not by 100
    // it is a leap year...
    if ((year % 4 == 0) && (year % 100 != 0))
        isLeap = true;
    else if (year % 400 == 0)
        isLeap = true;

    return isLeap;
}
```

The code in the method is preceded by a comment (e.g., starts with `/*` and ends with `*/`) that specifies the purpose of the method. Within the method's definition, lines starting with two slashes (e.g., `//`) are also comments. - (1) -

When this method is invoked to determine if a given year is a leap year, the flow of control in the computation starts at the first line of code in the method, where the `isLeap` variable is assigned `false` and continues through the method. The `isLeap` variable is used to store state as the method executes. To understand how the method works, it is helpful to consider the order in which statements of the method may execute. We refer to the order in which code executes as control-flow. When we consider how control flows within in a single method, we are considering intra-method control-flow. When one method must call another method to perform a desired computation, we must consider inter-method control-flow.

Footnotes:

- (1) - The code is simplified and will not run directly if typed into a Java compiler or environment.

Intra-method control-flow

The `isLeapYear` method includes two conditional statements that may alter which statements in the method execute depending upon the particular year being considered.

There are many different ways we might represent the intra-method control-flow for the `isLeapYear` method. We will use notation from a flowchart, an approach that has been used since the early days of computer science to plan programs. Figure 2 describes the four kinds of symbols that we will use in a flowchart for an intra-method control-flow model. Arrows connect these symbols to illustrate the flow of control.

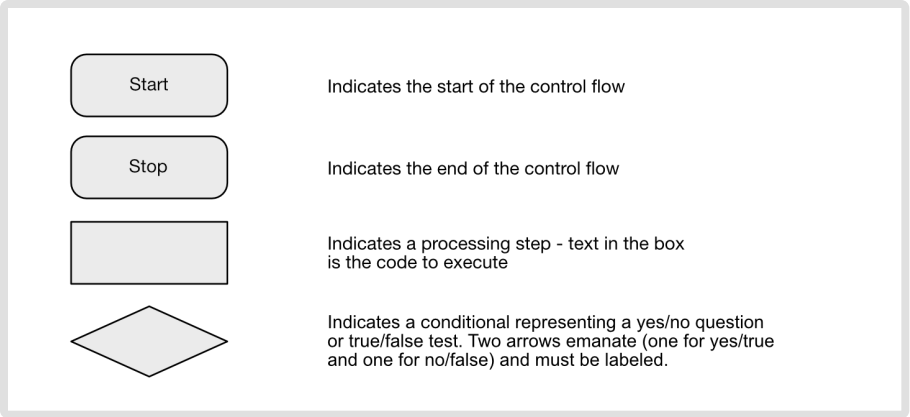


Figure 2: Flowchart Symbols

Figure 3 shows a flowchart for the `isLeapYear` method. In this flowchart, the first statement that can and must be executed is the assignment of the value `false` to the variable named `isLeap`. As our purpose in creating the flowchart is to understand the order in which statements may execute, we could simply number the statements in the method and use the numbers in the nodes of the flowchart. Instead, we use abbreviated forms of the statements to make it easier to map between the code in the method and the flowchart. The next statement to always execute is the conditional that tests whether the given year is divisible by 4 and not by 100. The arrow connecting the node that assigns `isLeap` the value `false` to the node that represents this conditional statement shows that control flows from one node to the next. The conditional statement represents a decision point or branch in the code and is represented by a diamond node. If the value of the conditional test is true, control next flows to the statement that sets the value of `isLeap` to `true`. Otherwise, control flows to the next conditional test to determine if the year is divisible by 400. No matter which route the flow of control takes through the various conditional tests, the final statement to be executed is the explicit return of the value of the `isLeap` variable as the value produced by the method.

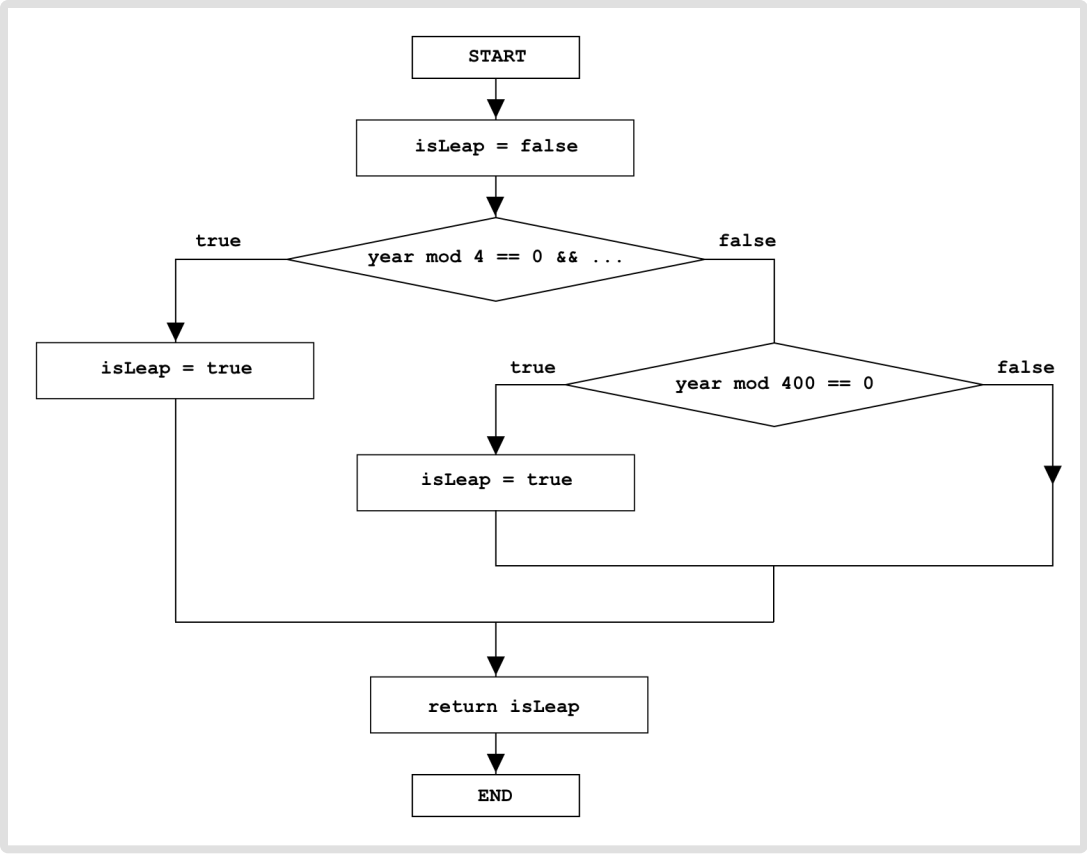
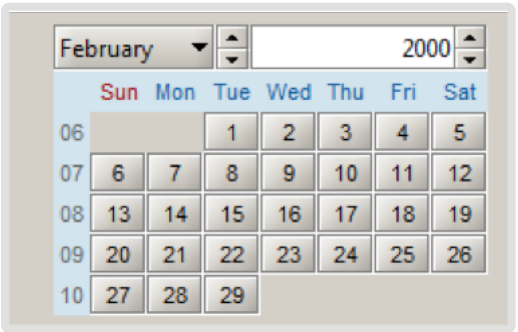


Figure 3: Flowchart for `isLeapYear` method

We will talk more in lecture about how to model code with a flowchart. After lecture, re-examine the flowchart and make sure you understand how to translate the `isLeapYear` method to the flowchart representation.

Inter-method control-flow



A single method by itself does not provide much interesting functionality to a user. A user might want to invoke the `isLeapYear` method to test whether a particular year is a leap year but it is unlikely. Instead, the user is likely to want the right dates to appear in a date chooser displayed by an application, such as the one shown to the right of this paragraph (from Kai Toedter, www.toedter.com/en/jcalendar/index.html).

The code below is a (greatly) simplified segment of Java code (from the site specified above) to support the drawing of the days of the specified year and month. - (2) -

```
/**
 * JDayChooser is a class for choosing a day
 *
 * @author Kai Toedter
 */
class JDayChooser {

   Calendar calendar;

   JDayChooser() {
      calendar = new Calendar();
      init();
   }

   void init() {
      drawDayNames();
      drawDays();
   }

   void drawDayNames() {
      int firstDayOfWeek = calendar.getFirstDayOfWeek();
      // generate and draw names
   }

   void drawDays() {
      // do a bunch of drawing
   }
}
```

This code outlines a partial definition for a `JDayChooser` Java class - (3) -, which is a graphical user interface (GUI) widget that supports the selection of a day in a displayed month of a specified year. This partial class definition includes one field, `calendar`. Recall that a field in a Java class stores state for an object constructed from the class. The statement `Calendar calendar` states that the field, `calendar` (note the lower case c) is of type `Calendar`. Following the definition of the field is a definition of a constructor that is used to create an object (or an "instance" in other terminology) of the `JDayChooser` class; in this case a `JDayChooser` object is one visible representation of a day chooser widget. Multiple objects can be created from a class, so a software application could show multiple `JDayChooser` objects at one time. Each `JDayChooser` object would have separate fields (e.g., in this case, separate copies of calendars). Three methods, `init`, `drawDayNames` and `drawDays`, are defined following the `JDayChooser` constructor. (As you look at the code for this class, think about how you might distinguish fields from constructors from methods.)

To start understanding how the `JDayChooser` class works, it is often helpful to first understand how the methods of the class work together, before diving into the details of how each method exactly works. This strategy is not that different than ones you might use when reading a chapter of a textbook, where you might first skim the chapter to see the topics and how they are organized before diving into the details. Just like you might read the chapter a few times to understand it, you may find it useful to read code in various iterations.

One way to depict the flow of control between methods is to use a method-based call graph, which is simply a diagram in which the nodes are methods and arrows depict possible flows of control between the methods. Figure 4 shows a method-based call graph for the `JDayChooser` constructor. If the methods are named well so that the name provides a correct clue to their function, a call graph can help a software developer understand — at an abstract level — how the software functions before diving into the details. Note that the call graph does not explicitly show where control starts or ends as a flowchart can. The diagram also does not indicate if a node always calls methods it points to or the order in which they are called.

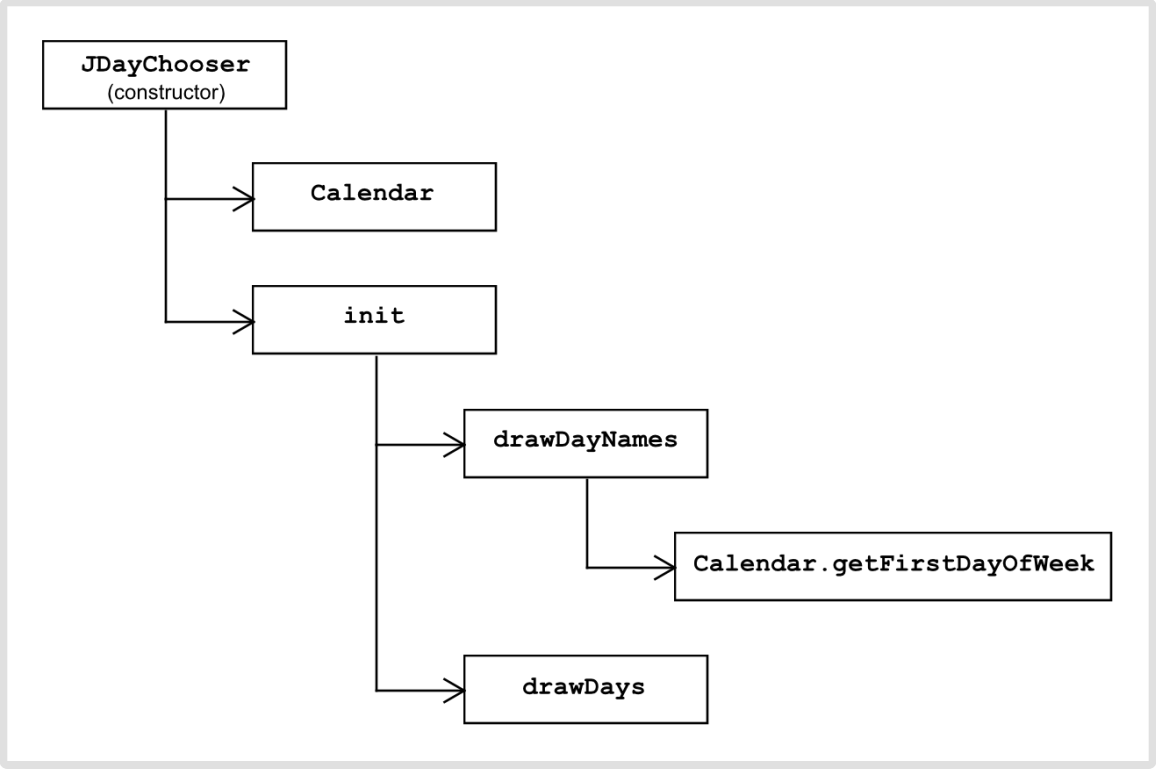


Figure 4: Method-based call graph

Inter-method control flow diagrams, such as the method-based call graph, are often used in the manner depicted in Figure 4; that is a call graph of a particular part of a software system may be determined to understand a part of the system at a time.

We will go over the details of creating a method-based call graph in class. In particular, we will be more careful about showing both the name of the class containing a method and the method name when drawing call graphs from this point on. Figure 4 is meant to capture and provide a simple case. After we go over call graphs in lecture, make sure you know how to create the above call graph from the given code.

Footnotes:

- (2) - As with the `isLeapYear` method, this code has been simplified to the point that it will not run. Method specifications and other comments have also been elided to keep the code short.

- (3) - Recall that an overview of classes and objects is provided in the Intermezzo section of the earlier readings on Data Abstraction.

References and Further Reading

The following sections of the Java tutorial describe some of the syntax that you've seen in the `isLeapYear` method. Read through these parts of the tutorial and then read through this document again.

- Equality, Relational and Conditional Operators: read the sections entitled The Equality and Relational Operators and The Conditional Operators http://download.oracle.com/javase/tutorial/java/nutsandbolts/op2.html
- The if-then and if-then-else statements http://download.oracle.com/javase/tutorial/java/nutsandbolts/if.html